pAIntry Backend API

Base URL: **https://<your-app>.onrender.com** (replace with the Render URL; local: **http://localhost:8000**)
Interactive docs: **<base>/docs**
CORS is open, so the frontend can call it directly. All responses are JSON.
The free Render instance sleeps after ~15 min idle. First request may take ~30 s.
Demo user id (seed): **665f00000000000000000001** (Maya). Household: **665f00000000000000000010**.

GET /health

Checks the server and database.

Response:

```
{"status": "ok"}
```

GET /products

Seeded products from the database.

Query param	Type	Required	Notes
category	string	no	e.g. "dairy_eggs", "produce"

Response: array of product objects, e.g.

```
{
  "_id": "665f...",
  "name": "Plain Greek Yogurt Nonfat",
  "brand": "Olympus",
  "category": "dairy_eggs",
  "size": 32,
  "unit": "oz",
  "barcode": "000000010012",
  "nutrition_per_serving": {
    "calories": 90, "protein_g": 16, "carbs_g": 6,
    "fat_g": 0, "fiber_g": 0, "sugar_g": 4
  },
  "prices": {"Store A": 5.49, "Store B": 4.99, "Store C": 5.99},
  "sample_data": true
}
```

GET /live-prices

Live prices from Kroger (stores nearest the user's ZIP).

Query param	Type	Required	Default
q	string	yes	
limit	int	no	5
zip	string	no	user zip or DEFAULT_ZIP

Example: GET /live-prices?q=greek%20yogurt&limit=3&zip=98105

Response: array of

```
{
  "name": "...", "brand": "...", "category": "...", "upc": "...",
  "size": "32 oz",
  "prices": {"QFC University Village (...)": 5.99},
  "cheapest_store": "...", "price": 5.99,
  "protein_g_per_package": 80, "cost_per_protein_g": 0.07,
  "allergens": ["milk"], "image": "https://...", "source": "kroger"
}
```

Errors: 503 if Kroger is unavailable.

GET /stores

Nearest Kroger-family stores (QFC, Fred Meyer, ...) to a ZIP.

Query param	Type	Required	Notes
zip	string	no	defaults to DEFAULT_ZIP (98105)

Response:

```
[
  {"name": "QFC University Village (2746 NE 45th St)", "location_id": "70500888"},
  {"name": "...", "location_id": "..."}
]
```

Errors: 503 if Kroger is unavailable.

GET /users/{user_id}

One user profile (allergies, goals, budget, phone, email, zip).

Response:

```
{
  "_id": "665f0000000000000000001",
  "name": "Maya",
  "phone": "+15555550101",
  "weekly_budget": 60,
  "allergies": ["peanuts"],
  "goals": {"protein_g_per_day": 120},
  "zip": "98105"
}
```

Errors: 400 invalid id, 404 not found.

GET /users/{user_id}/pantry

The user's pantry items, sorted by expires_at (soonest first).

Response:

```
[
  {
    "_id": "...",
    "user_id": "665f000000000000000000001",
    "name": "Baby Spinach",
    "quantity": 1,
    "unit": "bag",
    "expires_at": "2026-10-18T00:00:00Z"
  }
]
```

GET /users/{user_id}/pantry/food-groups

Classic food pyramid mix for the user's pantry. Primary percentages are **calorie share** by group vs targets: grains 30%, vegetables 25%, fruit 20%, protein & dairy 15%, fat & sweet 10%.

Response:

```
{
  "user_id": "665f000000000000000000001",
  "primary_metric": "calories",
  "pyramid_model": "classic",
  "pyramid_targets": {
    "grains": 30, "vegetables": 25, "fruits": 20,
    "proteins_dairy": 15, "sweets_fats": 10
  },
  "groups": {
    "grains": {
      "label": "Grains", "target_pct": 30, "pct": 42.1, "pct_calories": 42.1,
      "delta_from_target": 12.1, "status": "high", "grams_on_hand": 910, "calories_on_hand": 4200
    },
    "vegetables": { "...": "..." },
    "fruits": { "...": "..." },
    "proteins_dairy": { "...": "..." },
    "sweets_fats": { "...": "..." }
  },
  "combined_produce_pct": 28.5,
  "combined_produce_target_pct": 45,
  "pantry_balance_hint": ["Grains is 12 pts above the 30% pyramid target."],
  "lines": [
    {"name": "Baby Spinach", "group": "vegetables", "grams_on_hand": 142, "matched_product": true}
  ],
  "unclassified_grams": 0,
  "assumptions": ["..."]
}
```

Errors: 404 user not found.

POST /users/{user_id}/pantry

Add a pantry item. Returns the created item (201).

```
{
  "name": "Milk",
  "quantity": 1,
  "unit": "item",
  "product_id": null,
  "expires_at": "2026-10-20T00:00:00Z"
}
```

Only **name** is required.

Errors: 404 user not found.

DELETE /users/{user_id}/pantry/{item_id}

Remove one pantry line for that user (used when Finished / Trashed in the app).

Response:

```
{"deleted": true, "item_id": "665f000000000000000099"}
```

Errors: 400 invalid id, 404 user or pantry item not found.

GET /users/{user_id}/savings

Response:

```
{"user_id": "...", "total_saved": 12.34, "entries": 3}
```

POST /convince-me (main feature)

Decides whether to buy a product or swap to a cheaper or healthier one, and explains why with AI (Grok or Gemini; engine fallback).

Body (**user_id** plus one of **query** / **product_id** / **barcode**):

Field	Type	Default	Notes
user_id	string	required	
query	string	null	free text, e.g. "protein bar chocolate"
product_id	string	null	DB product id
barcode	string	null	
live	bool	true	use live Kroger prices for query
log	bool	true	false = don't save to history/savings (demos)

Can also pass **?log=false** in the URL.

Example:

```
{"user_id": "665f00000000000000000001", "query": "protein bar chocolate", "log": false}
```

Key response fields:

Field	Meaning
verdict	"Buy", "Skip", or "Swap"
headline	Short user-facing summary
reasons	AI-written bullets (numbers from engine)
better_alternative	{ "name", "store", "price" } or null
savings	Dollars saved when swapping
source	"grok", "gemini", or "engine" (fallback)
data_sources	Where prices/nutrition came from
engine	Raw facts: price_source, sample_data, expiring_soon, …
logged	Whether decision/ledger was saved

Example response:

```
{
  "verdict": "Swap",
  "headline": "Swap to Greek Yogurt Cup Vanilla, save $2.20",
  "reasons": ["Saves $2.20...", "$58.51 left in your weekly budget"],
  "better_alternative": {"name": "Greek Yogurt Cup Vanilla", "store": "Store B", "price": 1.29},
  "savings": 2.2,
  "source": "grok",
  "data_sources": ["Seeded sample prices (Store A/B/C)", "..."],
  "engine": { "product": { "name": "...", "price": 3.49 }, "draft_verdict": "Swap", "...": "..." },
  "logged": false
}
```

Errors: 400 missing product info or bad id, 404 user/product not found.

GET /users/{user_id}/list

Personal shopping list (solo; not the group cart).

Response:

```
{
  "user_id": "665f0000000000000000001",
  "items": [
    {
      "product_id": "...",
      "name": "Hummus Classic",
      "quantity": 1,
      "store": "Store B",
      "price": 3.49
    }
  ],
  "skipped": [{"name": "Whole Milk", "pantry_name": "Whole Milk"}],
  "total": 3.49,
  "count": 1
}
```

Errors: 404 user not found.

GET /households/{household_id}/cart

Current group cart for the household.

Response:

```
{
  "household_id": "665f00000000000000000010",
  "space_id": "demo",
  "items": [
    {
      "product_id": "...",
      "name": "Hummus Classic",
      "quantity": 1,
      "requesters": ["665f00000000000000000002", "665f00000000000000000001"],
      "store": "Store B",
      "price": 3.49
    }
  ],
  "skipped": [],
  "total": 3.49,
  "count": 1
}
```

Errors: 404 household not found.

POST /agent/message

Group iMessage / household cart agent. Mutates **group_carts** when not ignored. Receipt images supported when Perceptron is configured.

Body:

```
{
  "space_id": "demo",
  "sender": "+15555550101",
  "text": "add milk",
  "message_id": null,
  "image_base64": null,
  "image_mime_type": null,
  "space_type": null
}
```

Non-grocery chat: **ignored** true, **reply** null. Unknown sender phone defaults to Maya.

Response:

```
{
  "reply": "Skipped Whole Milk — you already have Whole Milk at home.",
  "cart": {"items": [], "skipped": [...], "total": 0, "count": 0},
  "source": "grok",
  "ignored": false,
  "logged": true,
  "space_id": "demo",
  "user_id": "665f00000000000000000001",
  "household_id": "665f00000000000000000010"
}
```

POST /agent/personal

1:1 DM personal planner. Mutates **personal_lists**. Only registered phones; unknown numbers get a signup message (**user_id** null).

Body: same shape as POST /agent/message.

Response:

```
{
  "reply": "Added Hummus Classic to your list ($3.49 at Store B).",
  "list": {"items": [...], "skipped": [], "total": 3.49, "count": 1},
  "source": "grok",
  "ignored": false,
  "logged": true,
  "space_id": "any;-;+15555550101",
  "user_id": "665f00000000000000000001"
}
```

POST /users/{user_id}/pantry/receipt

Debug receipt OCR flow without Perceptron (curl / tests).

```
{
  "ocr": {"items": [{"product": "Protein Bar Chocolate", "price": "$3.49"}]},
  "space_id": "debug"
}
```

Then continue in the same chat via POST /agent/personal with text: yes → yes → buy.

Response:

```
{
  "reply": "I found 1 item on your receipt. Reply YES to add them to your pantry.",
  "source": "engine",
  "lines": [...],
  "receipt_state": "await_pantry_confirm"
}
```

POST /login/start

Send SMS one-time code (Photon sidecar).

```
{"phone": "+15555550101"}
```

Response:

```
{"sent": true, "phone": "+15555550101"}
```

Errors: 400 invalid phone, 502 SMS failed.

POST /login/verify

```
{"phone": "+15555550101", "code": "123456"}
```

Response:

```
{"user_id": "...", "name": "", "phone": "+15555550101", "new_user": true}
```

POST /login/supabase

Exchange Supabase session for Mongo user (email / Google).

```
{"access_token": "<supabase access token>"}
```

Response:

```
{
  "user_id": "...",
  "name": "Maya",
  "email": "maya@example.com",
  "phone": null,
  "new_user": false
}
```

Errors: 500 Supabase not configured, 401 invalid token.

Frontend fetch examples

Convince Me:

```
const API = "https://<your-app>.onrender.com";
const USER_ID = "665f000000000000000000001";

const res = await fetch(`${API}/convince-me`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
```

```
  body: JSON.stringify({
    user_id: USER_ID,
    query: "protein bar chocolate",
    log: false,
  }),
});
const data = await res.json();
```

Pantry food groups:

```
const mix = await fetch(`${API}/users/${USER_ID}/pantry/food-groups`)
  .then(r => r.json());
```

Remove pantry item:

```
await fetch(`${API}/users/${USER_ID}/pantry/${ITEM_ID}`, {
  method: "DELETE",
});
```

Group cart (curl-style demo space):

```
await fetch(`${API}/agent/message`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({
    space_id: "demo",
    sender: "+15555550101",
    text: "add milk",
  }),
});
```

Supabase login → app user_id:

```
const { data: { session } } = await supabase.auth.getSession();
const login = await fetch(`${API}/login/supabase`, {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ access_token: session.access_token }),
}).then(r => r.json());
```